

Temario A

La librería de la UCI ha solicitado la elaboración de un sistema informático para la gestión de las ventas de algunas publicaciones expuestas en la Feria del Libro recién finalizada. Para ello el sistema almacena una lista con los datos de las publicaciones en venta. De cada publicación se conoce su ISBN, título, precio base y cantidad en existencia. Las publicaciones pueden ser Libros o revistas. De los libros además de los datos anteriores se almacena el nombre del autor, el número de la edición y la materia a la que pertenece. De las Revistas se almacena además la cantidad de artículos que contiene publicados.

El precio de las publicaciones es diferente para Libros y Revistas, se calcula de la siguiente manera:

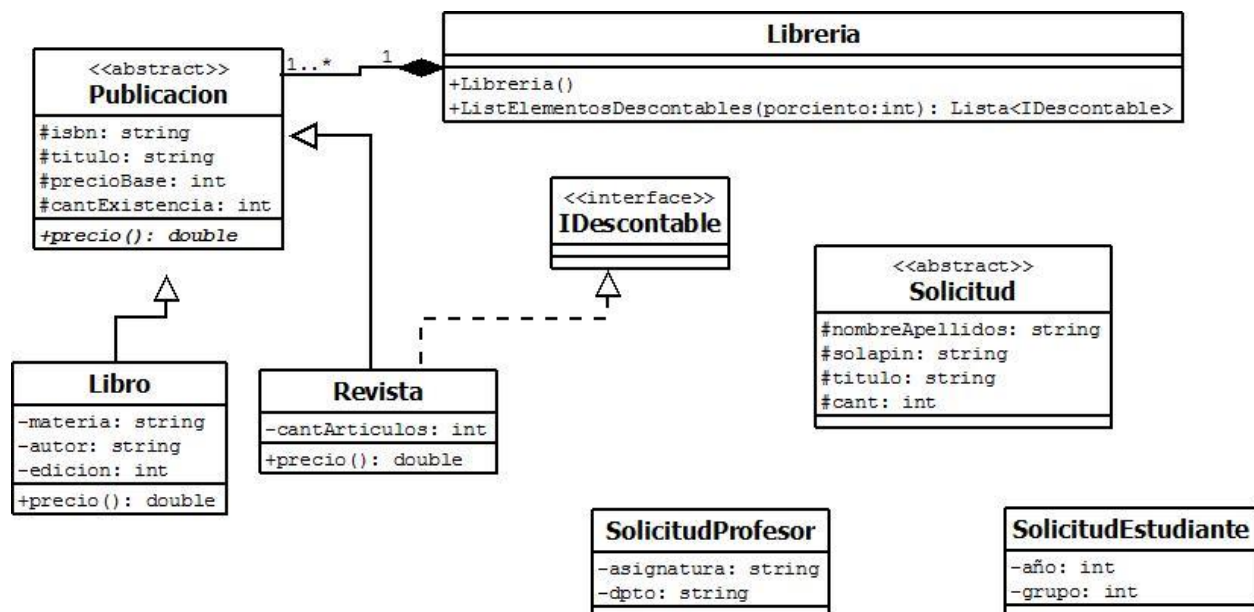
Libros: Si es la primera edición se calcula como el doble del precio base, en caso contrario sería el precio base + 15.

Revistas: precio base + 0.5*cantidad de artículos

Para solicitar la compra de una publicación los estudiantes y profesores deben llenar un formulario con su nombre y apellidos, solapín, el título de la publicación y la cantidad de ejemplares a comprar. Para los estudiantes además se deben almacenar los siguientes datos: año que cursa y grupo al que pertenecen. En el caso de los profesores el formulario a completar debe incluir además de los datos generales, la asignatura que imparte y el departamento docente al que pertenece. Cada solicitud se almacena en una lista de solicitudes.

La librería ofrece un por ciento de descuento en dependencia del tipo de persona que realice una solicitud de compra. Todas las solicitudes que se realicen obtienen inicialmente un 10% de descuento del precio si el solicitante compra más de tres publicaciones. En el caso de los estudiantes, se les realiza un descuento del 30% si están en 1ro, 2do o 3er año, en caso contrario se aplica el por ciento de descuento inicial. Algunas publicaciones también pueden recibir un 5% por ciento de descuento, pero este solo se aplica a las revistas con más de 50 ejemplares en existencia y menos de 6 artículos. Es de interés de la librería obtener un listado de los elementos descontables a los que se les aplicó un por ciento de descuento superior a un por ciento dado.

A continuación se observa una aproximación del diagrama de clases que modela la situación explicada anteriormente:



Preguntas a realizar

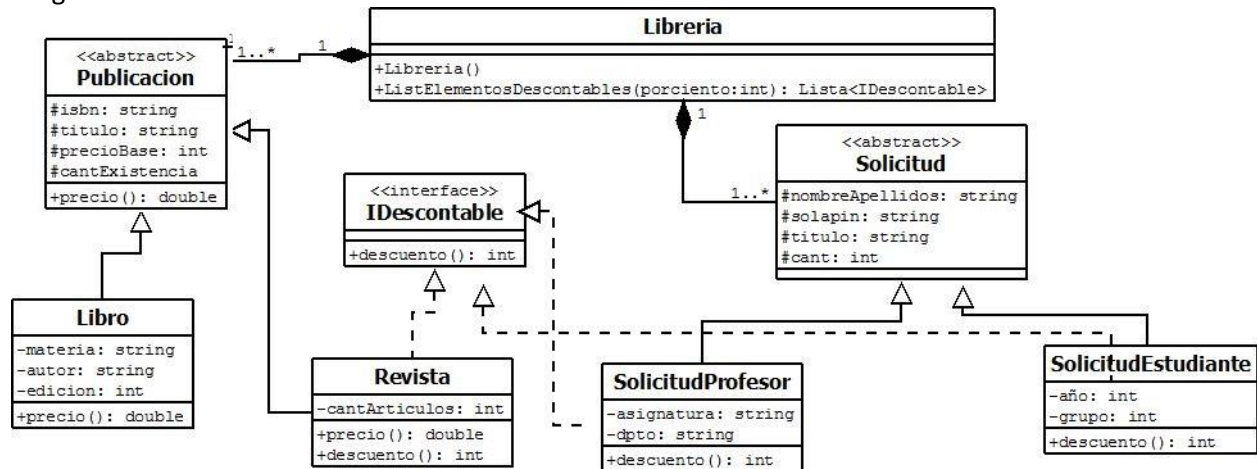
1. **Complete el diagrama de clases dado** con las relaciones y métodos que faltan, para que se ajuste al problema descrito y se garantice que el método **ListElementosDescontables** sea funcional. Asuma que los constructores de todas las clases se encuentran en el diagrama.
2. **Implemente**, en las clases correspondientes, los métodos polimórficos ubicados por usted en el diagrama de clases. Especifique como un comentario en la implementación, la clase a la que pertenece el método que implementa.
3. **Declare la clase controladora**, haciendo uso de la clase `Lista<T>` para la representación de las solicitudes y las publicaciones.
4. **Implemente en las clases correspondientes** los métodos necesarios para satisfacer las siguientes funcionalidades, teniendo en cuenta las situaciones excepcionales que puedan ocurrir:
 - a) Determinar si existen más libros que revistas en la librería.
 - b) Mostrar cuánto dinero recaudaría la librería por la venta de todos los libros de una materia específica dada por el usuario, asumiendo que no se aplica ningún tipo de descuento.
5. **Declare** una clase genérica nombrada *ListaAlterna* y que herede de la clase `Lista<T>`. Agregue a esta el método *Impares*, el cual retorna una lista con los elementos que se encuentran en las posiciones impares de la colección actual. Lance una excepción si la colección no tiene elementos en posiciones impares.
6. **Implemente** en las clases correspondientes, los métodos necesarios para salvar en el fichero texto "*publicaciones.txt*" y separados por un cambio de línea: el título de las publicaciones que están agotadas. Realice el manejo de excepciones correspondiente de modo que el fichero siempre se cierre, aún cuando se interrumpa la ejecución del método por ocurrir algún error en el proceso de escritura.

Clases para el trabajo con ficheros en Java

File	BufferedWriter	DataOutputStream
FileReader	FileInputStream	ObjectInputStream
FileWriter	FileOutputStream	ObjectOutputStream
BufferedReader	DataInputStream	

Respuestas del Temario A

Preg 1



Preg 2

// interface IDescontable

```
public abstract int descuento();
```

// clase Revista

```
public int descuento(){ if (cantExistencia>50 && cantArticulos<6) return 5; else return 0;}
```

// clase SolicitudProfesor

```
public int descuento(){ if(cant>3) return 10; else return 0; }
```

// clase SolicitudEstudiante

```
public int descuento(){
if(año==1 || año==2 || año==3) return 30;
else if(cant>3) return 10;
else return 0;
}
```

Preg 3

```
public class Librería{
private Lista<Solicitud> solicitudes ;
private Lista<Publicacion> publicaciones;
}
```

Preg 4 a)

```
public boolean masLibrosqueRevistas(){
int cantLib=0, cantRev=0;
for (int i=0; i<publicaciones.Longitud(); i++)
{
if(publicaciones.Obtener(i) instanceof Libro) cantLib++;
else cantRev++;
}
if(cantLib>cantRev) return true;
else return false;
}
```

Preg 4 b)

```

public double recaudoLibrosMateria(String materia)throws Exception{
    boolean existeMat=false;
    double recaudo=0;
    for(int i=0; i<publicaciones.Longitud(); i++)
    {
        if(publicaciones.Obtener(i) instanceof Libro)
        {
            if(((Libro)publicaciones.Obtener(i)).getMateria().compareTo(materia)==0)
            {
                existeMat=true;
                recaudo+=publicaciones.Obtener(i).precio();
            }
        }
    }
    if (!existeMat) throw new Exception("No existen ejemplares de esa materia");
    else return recaudo;
}

```

Preg 5

```

public class ListaAlterna<T> extends Lista<T> {
    public Lista<T> impares(){

        Lista<T> l=new Lista<>();
        for(int i=1; i< cantReal; i+=2)
        {
            l.Adicionar(elementos[i]);
        }
        return l;
    }
}

```

}

Preg 6

```

public void SalvarFichero() throws IOException {

    FileWriter file = new FileWriter("publicaciones.txt");
    BufferedWriter bf = new BufferedWriter(file);
    try{
        for(int i=0; i<publicaciones.Longitud(); i++)
            if(publicaciones.Obtener(i).getCantExistencia()==0)
                bf.write(publicaciones.Obtener(i).getTitulo() + "\r\n");
    }
    catch(IOException e){
    }
    finally{
        bf.close();
    }
}
}

```

Temario B

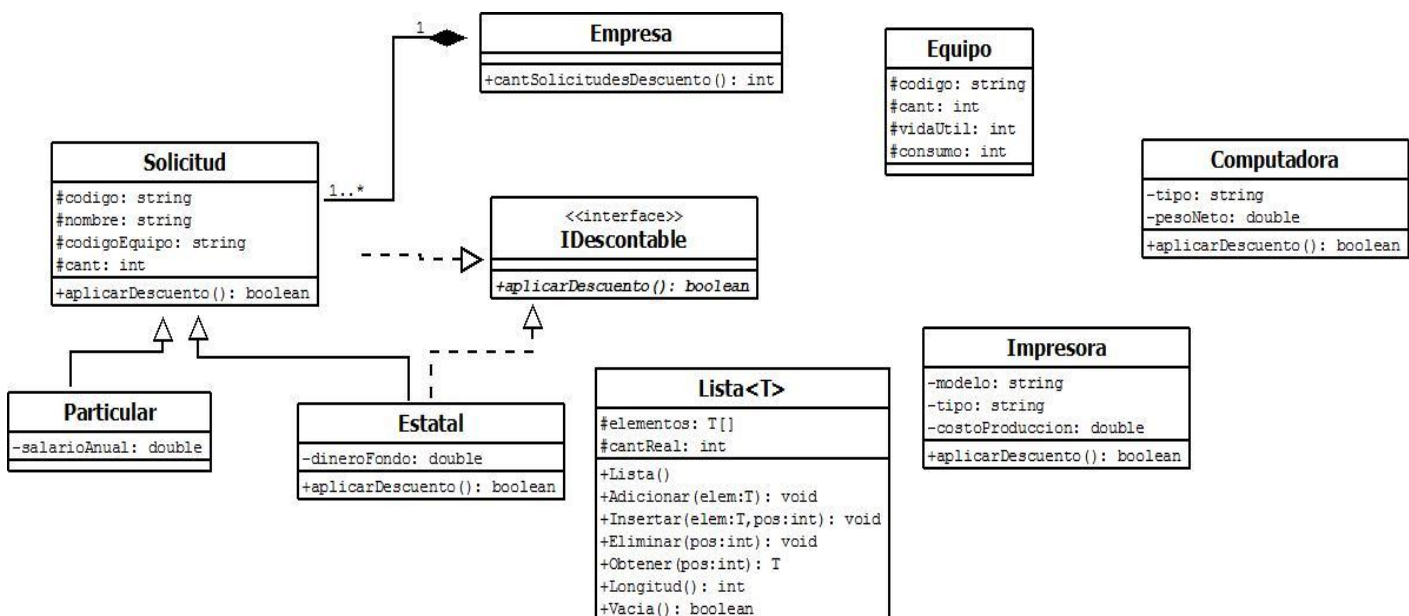
Una empresa productora de equipos de cómputo desea un sistema para gestionar y dar respuesta a las solicitudes de compra y reparación de equipos que recibe diariamente. El sistema debe llevar el control de los datos de los equipos del almacén que tiene en venta la institución en el momento actual, además de gestionar las solicitudes de compra que recibe. Para todos los equipos de cómputo, se tiene el código que lo identifica, la cantidad de equipos de ese tipo listos para la venta que tiene en existencia la empresa, la vida útil del equipo representado por un entero que indica la cantidad de años que puede durar el equipo, y el consumo en watts. Los equipos se clasifican en impresoras y computadoras. De las impresoras se controla además el modelo, el tipo (láser o tridimensional) y el costo de producción. De las computadoras su tipo (PC o Laptop) y el peso neto en kilogramos.

El precio de venta de los equipos varía de acuerdo a las características específicas de cada uno. Este se calcula, multiplicando la vida útil del equipo por el consumo en watts dividido entre 2.5. Además en caso de ser una impresora se le adiciona el valor del costo de producción, y si la impresora es tridimensional se adiciona el doble del costo de producción. Por otra parte si el equipo es una computadora al precio de venta se le adiciona el peso neto dividido por 1500.

Las solicitudes de compra, la empresa las registra en un listado de solicitudes a medida que las va recibiendo. De cada solicitud se registra el código que identifica la solicitud, el nombre del solicitante, el código del equipo, y la cantidad que desea el comprador. En caso de que la solicitud la realice una entidad estatal se registra además la cantidad de dinero que posee el fondo de la empresa y para las solicitudes particulares se debe registrar además el salario anual del comprador.

La empresa ofrece un descuento del precio de venta de los equipos en dependencia del tipo de solicitud, y el tipo de equipo. Todas las solicitudes que se realicen obtienen inicialmente un descuento del precio total si el solicitante compra más de dos equipos. En el caso de las solicitudes estatales además se les realiza el descuento si el fondo de la empresa es menor a los 3000 pesos. Algunos equipos también pueden recibir un por ciento de descuento, pero este solo se aplica a las impresoras laser con menos de 5 años de vida útil, y a las computadoras con mas de 2.5 kilos de peso neto. Es de interés de la empresa conocer a cuantas solicitudes se les aplicó descuento del precio de venta.

A continuación se observa una aproximación del diagrama de clases que modela la situación explicada anteriormente:



Preguntas a realizar

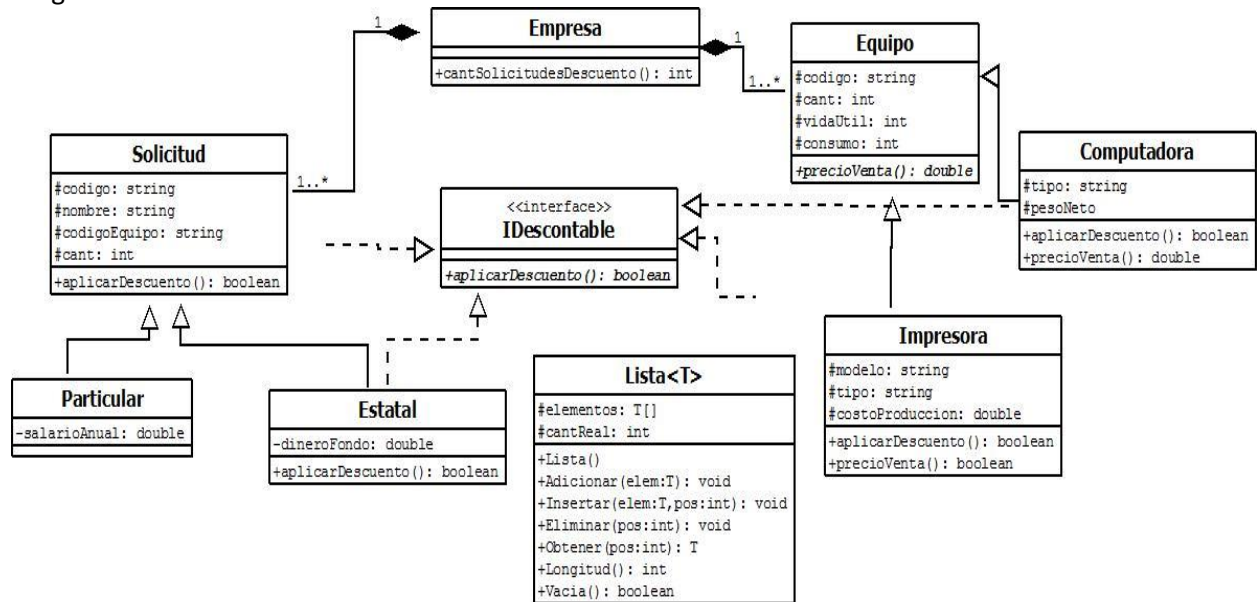
1. **Complete el diagrama de clases dado** con las relaciones y métodos que faltan, para que se ajuste al problema descrito y se garantice que el método **cantSolicitudesDescuento** sea funcional. Asuma que los constructores de todas las clases se encuentran en el diagrama.
2. **Implemente**, en las clases correspondientes, los métodos polimórficos ubicados por usted en el diagrama de clases. Especifique como un comentario en la implementación, la clase a la que pertenece cada método que implementa.
3. **Declare la clase controladora**, haciendo uso de la clase `Lista<T>` para la representación de las solicitudes y los equipos de cómputo.
4. **Implemente en las clases correspondientes** los métodos necesarios para satisfacer las siguientes funcionalidades, teniendo en cuenta las situaciones excepcionales que puedan ocurrir:
 - a) Dado el código de un equipo devolver la cantidad total de este que la empresa debe vender por que ha sido solicitado en el listado de solicitudes.
 - b) Devolver un listado de las solicitudes estatales que han obtenido descuento.
5. **Declare** una clase genérica nombrada *ListaAlternativa* y que herede de la clase `Lista<T>`. Agregue a esta el método *EliminarAlternos*, el cual elimina de la lista los elementos que se encuentran en las posiciones impares de la colección actual. Lance una excepción si la colección no tiene elementos a eliminar.
6. **Implemente** en las clases correspondientes, los métodos necesarios para restaurar en una lista los 50 equipos de computo almacenados en el fichero binario *"equipos.obj"*. Realice el manejo de excepciones correspondiente de modo que el fichero siempre se cierre, aún cuando se interrumpa la ejecución del método por ocurrir algún error en el proceso de lectura.

Clases para el trabajo con ficheros en Java

File	BufferedWriter	DataOutputStream
FileReader	FileInputStream	ObjectInputStream
FileWriter	FileOutputStream	ObjectOutputStream
BufferedReader	DataInputStream	

Respuestas del Temario B

Preg 1



Preg 2

//clase Equipo

```
public double precioVenta(){
    return vidaUtil*consumo/2.5;
}
```

//clase Impresora

```
@Override
public double precioVenta(){
    double c=costoProduc;
    if(tipo.compareTo("tridimensional")==0) c=2*costoProduc;
    return super.precioVenta() + c;
}
```

//clase Computadora

```
@Override
public double precioVenta(){
    return super.precioVenta() + pesoNeto/1500;
}
```

Preg 3

```
public class Empresa {
    private Lista<Solicitud> solicitudes;
    private Lista<Equipo> equipos;
    public Empresa(){
        solicitudes=new Lista<Solicitud>();
        equipos=new Lista<Equipo>();
    }
}
```

Preg 4 a)

```

public int cantTotalEquipo(String codigo)throws Exception{

    boolean existeEquipo=false;
    int cont=0;
    for(int i=0; i<solicitudes.Longitud(); i++){
        if(solicitudes.Obtener(i).getCodigo().compareTo(codigo)==0) {
            existeEquipo=true;
            cont+=solicitudes.Obtener(i).getCant();
        }

    }
    if (existeEquipo) return cont;
    else throw new Exception("No ha sido solicitada la compra de este equipo");
}

```

Preg 4 b)

```

public Lista<Solicitud> listadoEstatalesDescuento() throws Exception{
    Lista<Solicitud> list=new Lista<Solicitud>();

    for(int i=0; i<solicitudes.Longitud(); i++){
        if(solicitudes.Obtener(i) instanceof Estatal )

        {
            if(solicitudes.Obtener(i).aplicarDescuento())
                list.Adicionar(solicitudes.Obtener(i));
        }
    }
    if (list==null) throw new Exception("No se aplicado descuento a ninguna empresa estatal");
    else return list;

}

```

Preg 5

```

public class ListaAlterna<T> extends Lista<T>{

    public ListaAlterna(){
        super();
    }
    public void EliminarAlternos() throws Exception{
        if (Vacía()) throw new Exception("Lista vacía");
        for(int i=Longitud(); i>=0; i--){
            if(i%2!=0) Eliminar(i);
        }
    }
}

```

Preg 6

```

public Lista<Equipo> leerFichero() throws IOException, ClassNotFoundException {

    Lista<Equipo> list=new Lista<Equipo>();
    FileInputStream fs = new FileInputStream("equipos.obj");
    ObjectInputStream os = new ObjectInputStream(fs);
    try{

```



```

        for(int i=0; i<50; i++)
            list.Adicionar((Equipo) os.readObject());

    }
    catch(IOException e){

    }
    finally{
        os.close();
    }

    return list;
}

```

Temario C

Una Empresa de Promociones turísticas, desea mantener información sobre las estructuras de alojamiento (Hoteles) para turistas con que cuenta, de modo tal que sus clientes puedan planear sus vacaciones de acuerdo a sus posibilidades. La empresa cuenta con tres tipos alojamientos, los alojamientos se identifican por un código y un nombre, estos pueden ser de 3, 4 o 5 estrellas. Independientemente del tipo de hotel controla su cantidad de habitaciones de cada tipo (personal, pareja, familiar) y cantidad de pisos, además por cada tipo se controlan:

Hoteles 3 estrellas: si poseen servicio de habitación.

Hoteles 4 estrellas: tipo de gimnasio.

Hoteles 5 estrellas: capacidad del restaurante y nombre del mismo.

Los gimnasios pueden ser de tipo A o de tipo B.

El precio de una habitación es fijo y debe calcularse de acuerdo a las características del Hotel al que pertenezca siguiendo la siguiente fórmula:

Hoteles 3 estrellas: $5 * \text{cantidad de pisos} + (10\% \text{ de la capacidad del hotel})$

Hoteles 4 estrellas: $10 * \text{cantidad de pisos} + (10\% \text{ de la capacidad del hotel}) + (\text{VAR})$

Hoteles 5 estrellas: $15 * \text{cantidad de pisos} + (10\% \text{ de la capacidad del hotel}) + (\text{VAG})$

Donde:

- VAR (valor agregado por restaurante): \$10 si la capacidad del restaurante es de menos de 30 personas, \$30 si está entre 30 y 50 personas y \$50 si es mayor de 50 personas.
- VAG (valor agregado por gimnasio): \$50 si el gimnasio es de tipo A y \$30 si es de tipo B.

La empresa tiene un plan de mejora de sus hoteles por año, el sistema consiste en aumentar la capacidad de sus instalaciones si es necesario, para ello se tiene en cuenta la *necesidad* de habitaciones. La *necesidad* se calcula teniendo en cuenta las fórmulas que se muestran a continuación:

Hoteles 3 estrellas: $(10\% \text{ de cantidad de } 4 \text{ habitaciones} - (\text{cantidad de habitaciones} / \text{cantidad de pisos})) + 1$

Hoteles 4 estrellas: $(10\% \text{ de cantidad de habitaciones} - (\text{cantidad de habitaciones} / \text{cantidad de pisos})) / 2$

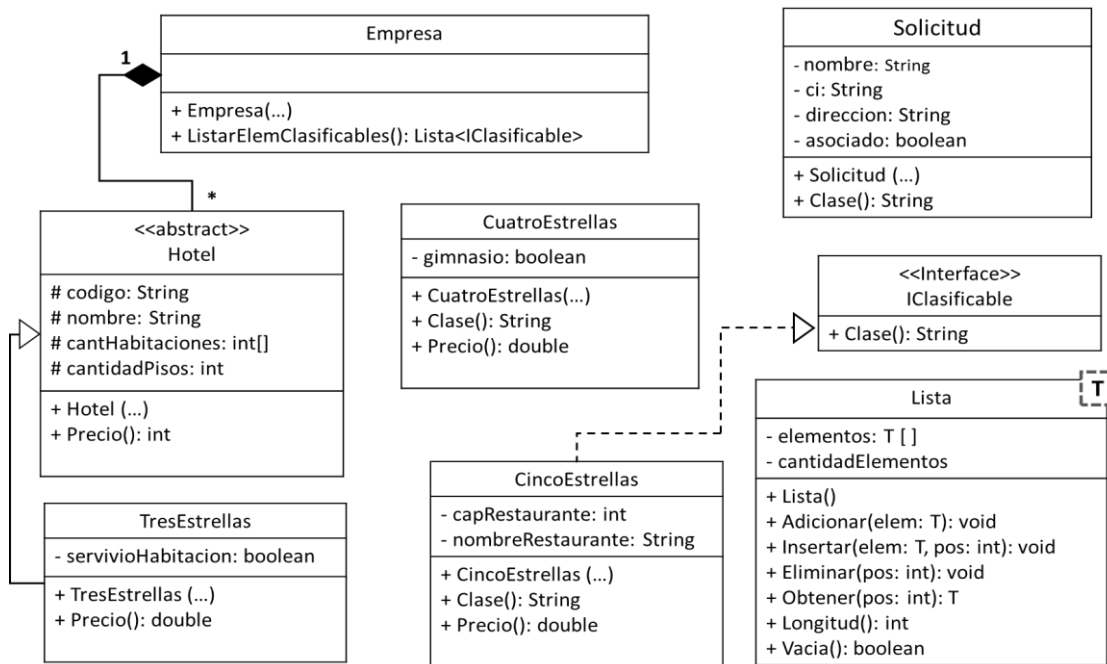
Hoteles 5 estrellas: $10\% \text{ de cantidad de habitaciones} - (\text{cantidad de habitaciones} / \text{cantidad de pisos})$

Para hacer una solicitud de reserva se llena un modelo que cuenta con los siguientes datos: Id de la solicitud, cuota máxima diaria (cantidad que puede pagar diariamente), el tipo de habitación que desea, cantidad y categoría del hotel.

Es interés de la empresa llevar un registro con la *clase* de las habitaciones de los hoteles 4 y 5 estrellas, que puede ser A,B o C de igual manera a partir de la solicitud se puede conocer la clase del visitante. Es por ello

que cuentan con una funcionalidad que les permite determinar dicha clase. La biblioteca por su parte, debe permitir obtener el listado de los elementos que son clasificables.

Teniendo en cuenta la descripción anterior y el diagrama de clase que se modela a continuación, responda los siguientes incisos:



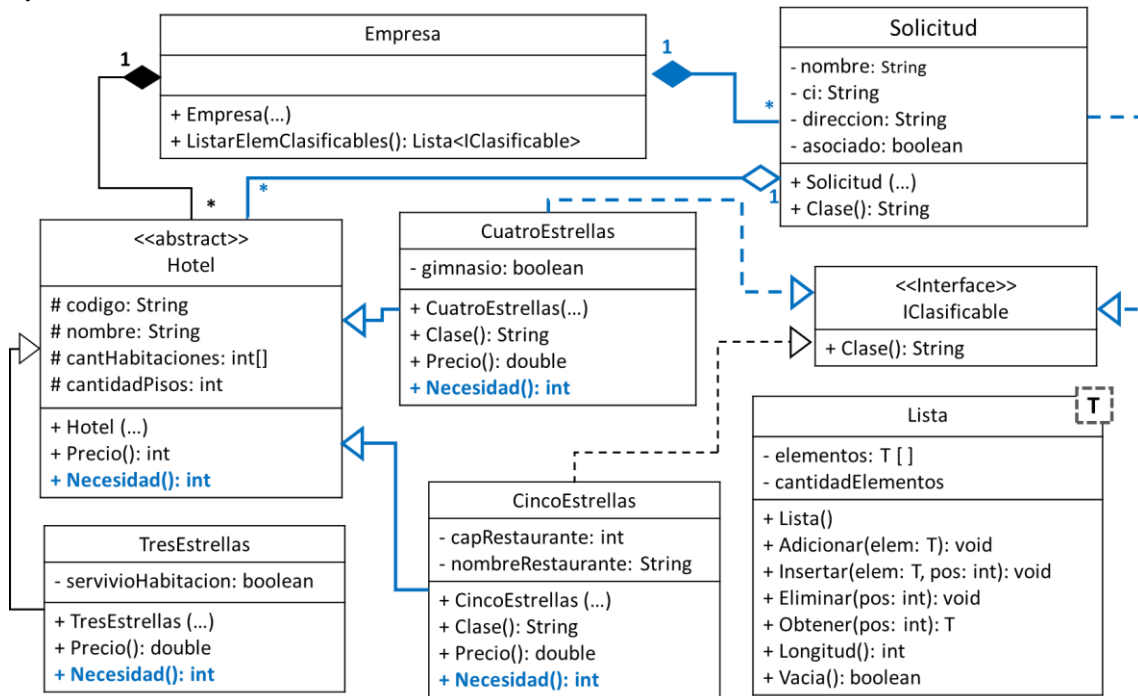
1. Complete el diagrama de clases dado con las relaciones y métodos que faltan, para que se ajuste al problema descrito.
2. Implemente, en las clases correspondientes, los métodos polimórficos ubicados por usted en el diagrama de clases.
3. Declare la clase controladora, haciendo uso de la clase `Lista<T>` para la representación de los Materiales.
4. Implemente en la clase correspondiente:
 - a. El método ***HotelMayorGanancia():Hotel***, el cual retorna cuál de los hoteles reporta mayor ganancia, teniendo en cuenta que la ganancia de un hotel está determinada por: (la cantidad de habitaciones * precio de las habitaciones).
 - b. El método ***ListarHotelesClaseA():Lista<Hotel>***, el cual retorna un listado de los **hoteles 5 Estrellas** con *clase A*.
5. Declare una clase genérica nombrada *ListaAux* y que herede de la clase `Lista<T>`. Implemente el método ***PosicionesPares(pos: int):Lista<T>*** que retorna los elementos de las posiciones pares a partir de una posición dada. Lance una excepción si la colección esta vacía o si la posición está fuera de rango para ello cree una nueva clase de excepciones llamada **PosFueraRangoException** y suma que existe una clase excepción **ListaVacíaException**.
6. Implemente en las clases correspondientes, los métodos necesarios para salvar en un fichero binario "hoteles.bin" el listado de hoteles, separando por un cambio de línea cada hotel. Realice el manejo de excepciones correspondiente de modo que el fichero siempre se cierre, aun cuando se interrumpa la ejecución del método por ocurrir algún error en el proceso de escritura.

Clases para el trabajo con ficheros en Java

File	FileReader	FileWriter	BufferedReader	FileInputStream
BufferedWriter	FileOutputStream	DataInputStream	ObjectInputStream	ObjectOutputStream
DataOutputStream				

Respuestas del Temario C

1)



2) **Clase Hotel**

```
public abstract int Necesidad();
```

Clase TresEstrellas

```
public int Necesidad ()
{ return (0.1*cantHabitaciones - cantHabitaciones / cantidadPisos)+1 ; }
```

Clase CuatroEstrellas

```
public int Necesidad ()
{ return (0.1*cantHabitaciones - cantHabitaciones / cantidadPisos)/2 }
```

Clase CincoEstrellas

```
public int Necesidad ()
{ return 0.1*cantHabitaciones - cantHabitaciones / cantidadPisos; }
```

```
3) public class Empresa {
    Lista<Hotel> hoteles;
    Lista<Solicitud> solicitudes;
}
```

4) a) **Clase Hotel**

```
public int Ganancia(){ return canthabitaciones*Precio(); }
```

Clase Empresa

```
public Hotel HotelMayorGanancia (){
    Hotel h = hoteles.Obtener(0);
    for(int i = 1; i < hoteles.Longitud(); i++){
        if(hoteles.Obtener(i). Ganancia () > h. Ganancia ())
            h = hoteles.Obtener(i);
    }
    return h; }
```

```

b) public Lista<Hotel>ListarHotelesClaseA (){
    Lista< Hotel > claseA = new Lista< Hotel > ();
    for(int i = 1; i < hoteles.Longitud(); i++)
        if(hoteles.Obtener(i) instanceof IClasificable)
            if ( ((IClasificable) hoteles.Obtener(i)).Clase().equals("A"))
                { claseA.Adicionar(hoteles.Obtener(i)) }
    return claseA;
}

```

```

5) public class ListaAux<T> extends Lista<T>{
    public ListaAux () { super(); }
    public Lista<T> PosicionesPares (int pos) throws PosFueraRangoException, ListaVacíaException {

        if (Vacía()) throw new ListaVacíaException();
        if (pos < -1 || pos >=Longitud()) throw new PosFueraRangoException ();

        Lista<T> pares = new Lista<T> ();
        if (pos%2 != 0) pos++;
        for(int i = pos; i < Longitud(); i+=2)
            pares.Adicionar(obtener(i));
        return pares;
    }
}

```

```

public class PosFueraRangoException extends Exception{
    public PosFueraRangoException () { super("Posicion fuera de rango"); }
}

```

6) Clase Hotel

```

public class Hotel implements Serializable {..}

```

Clase Empresa

```

public void SalvarHoteles () throws FileNotFoundException, IOException {
    FileOutputStream fos = new FileOutputStream("hoteles.bin");
    ObjectOutputStream dos = new ObjectOutputStream (fos);
    try {
        for (int i = 0; i < hoteles.Longitud(); i++) {
            dos.writeChars("\n");
            dos.writeObject(hoteles.Obtener(i));
        }
    } finally { dos.close(); }
}

```